FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	*Benedict Carl R. Roxas*
Section:	*AN42*
Professor:	*Ms. Crisola G. Tan*
Date:	*Oct 5, 2026*

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. *Construct a simple knowledge base using Python data structures.*
2. *Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.*
3. *Build a simple Case-Based Reasoning (CBR) system using similarity matching.*
4. *Compare how RBR and CBR behave when solving the same type of problem.*
5. *Document observations and insights on KR, RBR, and CBR.*

Materials Needed

1. Hardware

- *A laptop or desktop computer*
- *Stable internet connection (for research, documentation, submission)*

2. Software

A programming environment:

- *Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)*
- *Any text editor (Notepad++, Sublime, VS Code) if needed*

3. File Formats & Tools

Word processor for documentation:

- *MS Word, Google Docs, or LibreOffice*
- *Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)*

General Instructions

1. *This* **Technical Assessment (TA3)** *must be completed by Pair.*
2. *Include clear comments in your code to describe its logic.*
3. *Each program should display the correct input, process, and output.*
4. *Ensure that your script displays clear and readable outputs.*
5. *Screenshot of program output for RBR*
6. *Screenshot of program output for CBR*
7. *Reflection answers*
8. *Short description of your knowledge base and use case*
9. *Compile all Python codes and corresponding outputs into one PDF file.*
10. *Save it as* **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. *Submit the* **python file** *and* **PDF** *file.*

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- *5 facts about an object (e.g., weather, appliances, students, medical symptoms)*
- *3 categories (e.g., temperature, status, presence, actions)*

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. *Reads the knowledge base*
2. *Checks each rule*
3. *Returns all actions inferred*

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, *each containing:*

- *A set of problem features*
- *A matching solution*

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. *Which system (RBR or CBR) was easier to implement? Why?*
2. *Which is more flexible in solving unstructured problems?*
3. *Which is better for situations with continuously changing data?*
4. *Examples of real-world applications for both.*

Your Code - Laboratory Tasks ⇩

```
# IMPLEMENTING KNOWLEDGE REPRESENTATION, RBR, AND CBR

# PART 1 - KNOWLEDGE REPRESENTATION

facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
    "is_cloudy": True,
```

```
    "wind_speed": 20,
    "status": "hot"
}

print("KNOWLEDGE BASE")
for key, value in facts.items():
    print(key, ":", value)


# PART 2 - RULE-BASED REASONING

def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"


def rule_humid(facts):
    if facts["humidity"] > 80:
        return "Drink more water"


def rule_cloudy(facts):
    if facts["is_cloudy"]:
        return "Bring an umbrella"


def rule_windy(facts):
    if facts["wind_speed"] > 15:
        return "Be careful of strong winds"


rules = [
    rule_hot,
    rule_humid,
    rule_cloudy,
    rule_windy
]

print("\nRULE-BASED REASONING ACTIONS:")
```

```
actions = []

for rule in rules:
    result = rule(facts)

    if result:
        actions.append(result)
        print("-", result)


# PART 3 - CASE-BASED REASONING

case_base = [
    {
        "problem": {
            "temperature": 35,
            "is_raining": False,
            "humidity": 80
        },
        "solution": "Stay indoors and drink plenty of water."
    },
    {
        "problem": {
            "temperature": 28,
            "is_raining": True,
            "humidity": 85
        },
        "solution": "Bring an umbrella and avoid flooded areas."
    },
    {
        "problem": {
            "temperature": 30,
            "is_raining": False,
            "humidity": 70
        },
        "solution": "Stay hydrated and avoid too much outdoor activity."
    }
]


new_case = {
```

```
    "temperature": int(input("\nEnter current temperature: ")),
    "is_raining": input("Is it raining? (yes/no): ").lower() == "yes",
    "humidity": int(input("Enter humidity: "))
}


# Task 3B - Similarity Assessment

def calculate_similarity(new_problem, old_problem):
    score = 0

    if new_problem["temperature"] >= 30 and old_problem["temperature"] >= 30:
        score += 1

    if new_problem["is_raining"] == old_problem["is_raining"]:
        score += 1

    if abs(new_problem["humidity"] - old_problem["humidity"]) <= 10:
        score += 1

    return score


best_case = None
best_score = -1

print("\nCBR SIMILARITY SCORES:")

for i, case in enumerate(case_base, 1):
    score = calculate_similarity(new_case, case["problem"])

    print("Case", i, ":", score, "/ 3")

    if score > best_score:
        best_score = score
        best_case = case


# Task 3C - Reuse
```

```
print("\nRETRIEVED CASE:")
print(best_case["problem"])
print("Similarity Score:", best_score, "/ 3")

print("\nREUSED SOLUTION:")
print(best_case["solution"])


# Task 3D - Revise

revised_solution = input(
    "\nEnter a revised solution, or press Enter to keep the original: "
)

if revised_solution.strip() == "":
    final_solution = best_case["solution"]
else:
    final_solution = revised_solution


print("\nFINAL SOLUTION:")
print(final_solution)


# Task 3E - Retain

new_case_with_solution = {
    "problem": new_case,
    "solution": final_solution
}

case_base.append(new_case_with_solution)

print("\nNew case saved successfully!")

print("\nUPDATED CASE BASE:")

for i, case in enumerate(case_base, 1):
    print("Case", i)
    print("Problem:", case["problem"])
```

```
    print("Solution:", case["solution"])
    print()
```

```
KNOWLEDGE BASE
temperature : 32
is_raining : False
humidity : 85
is_cloudy : True
wind_speed : 20
status : hot

RULE-BASED REASONING ACTIONS:
- Turn on the aircon
- Drink more water
- Bring an umbrella
- Be careful of strong winds

Enter current temperature: 33
Is it raining? (yes/no): no
Enter humidity: 82

CBR SIMILARITY SCORES:
Case 1 : 3 / 3
Case 2 : 1 / 3
Case 3 : 2 / 3

RETRIEVED CASE:
{'temperature': 35, 'is_raining': False, 'humidity': 80}
Similarity Score: 3 / 3

REUSED SOLUTION:
Stay indoors and drink plenty of water.

Enter a revised solution, or press Enter to keep the original:

FINAL SOLUTION:
Stay indoors and drink plenty of water.

New case saved successfully!
```

```
UPDATED CASE BASE:
Case 1
Problem: {'temperature': 35, 'is_raining': False, 'humidity': 80}
Solution: Stay indoors and drink plenty of water.

Case 2
Problem: {'temperature': 28, 'is_raining': True, 'humidity': 85}
Solution: Bring an umbrella and avoid flooded areas.

Case 3
Problem: {'temperature': 30, 'is_raining': False, 'humidity': 70}
Solution: Stay hydrated and avoid too much outdoor activity.

Case 4
Problem: {'temperature': 33, 'is_raining': False, 'humidity': 82}
Solution: Stay indoors and drink plenty of water.
```

Program Description:

The knowledge base contains weather information such as temperature, rain, humidity, cloudiness, wind speed, and weather status. The use case is a simple weather assistant that uses RBR to give actions based on the current weather and CBR to find solutions from similar past weather situations.

Reflection Answers:

1. *RBR was easier to implement because the rules are simple IF-THEN statements. I only needed to create conditions and actions for each rule.*
2. *CBR is more flexible for unstructured problems because it can use past cases that are similar to the current problem, even when the problem does not follow fixed rules.*
3. *CBR is better for continuously changing data because new cases can be added to the case base. This allows the system to learn from new situations.*

4. *RBR can be used in medical diagnosis, troubleshooting systems, and weather alert systems. CBR can be used in customer support, medical case analysis, and diagnosing technical problems based on past cases.*